

Universidade do Mindelo

Trabalho 02

Escalabilidade em Sistemas Distribuídos

Relatório técnico

Repositório: API-monolitica

Stack: Python, Flask, Gunicorn, SQLAlchemy, Postgres, Docker Compose

1. Introdução

Este trabalho dá continuidade ao Trabalho 01, onde foi implementada uma arquitetura monolítica e uma versão com microserviços comunicando por HTTP real através de um API Gateway. O objetivo agora é evoluir o sistema para suportar um maior volume de requisições, respondendo a três perguntas: o que acontece quando o número de utilizadores aumenta, como escalar os serviços e quais os limites de cada abordagem.

O projeto sofreu três evoluções principais em relação ao Trabalho 01: (i) cada componente passou a correr num container próprio orquestrado por Docker Compose, sob um servidor WSGI de produção (Gunicorn); (ii) o serviço de pagamento foi replicado em três instâncias independentes, com um load balancer round-robin manual implementado dentro do monólito; (iii) foi adicionada persistência em Postgres com suporte a chave de idempotência, permitindo que retries sob carga não criem pedidos duplicados.

2. Fundamentos teóricos

2.1 Escalabilidade: vertical vs horizontal

Escalabilidade é a capacidade de um sistema absorver mais carga sem degradar significativamente o seu desempenho. A escala vertical consiste em aumentar os recursos de uma única máquina (mais CPU, mais RAM, disco mais rápido). É simples mas tem um teto físico e custo crescente, além de não tolerar falhas: se a máquina cai, o serviço cai com ela.

A escala horizontal consiste em adicionar mais instâncias do mesmo serviço, distribuindo as requisições entre elas. Não tem o mesmo teto físico, dá tolerância a falhas (uma instância pode cair sem derrubar o sistema), mas exige que o serviço seja stateless ou que o estado seja externalizado, e exige um mecanismo de distribuição: o load balancer.

2.2 Load balancing

Um load balancer é um componente que recebe requisições e decide a qual réplica encaminhar cada uma. As estratégias clássicas são round-robin (alterna entre as réplicas em sequência), least-connections (envia para a réplica com menos conexões abertas) e weighted (atribui pesos a réplicas com capacidades diferentes). Round-robin é o mais simples e o que foi escolhido neste trabalho por ser suficiente para cargas homogêneas como a do load test, onde cada requisição custa aproximadamente o mesmo.

2.3 Replicação de serviços

Replicação significa correr múltiplas cópias idênticas do mesmo serviço. Para que a replicação funcione, o serviço precisa de ser stateless ou ter o seu estado num backend partilhado (base de dados, cache distribuída). No nosso projeto, o serviço de pagamento é trivialmente stateless: recebe um pedido e devolve uma decisão. Por isso, replicá-lo é seguro: qualquer instância dá a mesma resposta para a mesma entrada.

2.4 Stateless vs Stateful

Um serviço stateless não guarda informação entre requisições — a resposta depende apenas dos dados do pedido. Um serviço stateful guarda informação (sessões, cache local, ficheiros) e por isso requisições do mesmo cliente podem ter de ser encaminhadas para a mesma

réplica (sticky sessions) ou o estado precisa ser externalizado. No projeto, o gateway e o serviço de pagamento são stateless. O estado persistente fica num componente dedicado: o Postgres.

2.5 Bottlenecks em sistemas distribuídos

Os gargalos típicos surgem quando um componente partilhado se torna o ponto único que limita o throughput do sistema. Os mais comuns: a base de dados (especialmente em escritas, devido a locks), o load balancer (se for um único nó), chamadas síncronas entre serviços (uma chamada lenta segura threads/workers a montante e propaga lentidão em cascata) e dependências externas sem timeout. Neste projeto, mitigamos as chamadas síncronas com timeout curto e retries com backoff, e evitámos a saturação de workers replicando o pagamento.

3. Implementação prática

3.1 Visão geral da arquitetura

Antes (Trabalho 01) — uma só instância de cada componente:

```
Cliente -> Gateway -> Monolito -> PaymentService
           (Flask dev)      (Flask dev)
```

Depois (Trabalho 02) — gateway escalado com Gunicorn, pagamento replicado, balanceador no monólito, persistência em Postgres:

```

                                     +-- payment-1 (gunicorn x2)
Cliente -> Gateway -> Monolito --+-- payment-2 (gunicorn x2)
                               (gunicorn x4) (RR LB) +-- payment-3 (gunicorn x2)
                               |
                               v
                             Postgres
```

3.2 Parte A — Escala no monólito

A primeira limitação encontrada no Trabalho 01 era óbvia: o sistema corria sob o servidor de desenvolvimento do Flask, que serve uma requisição de cada vez por thread e não é recomendado para produção. Substituímos o servidor de desenvolvimento por Gunicorn, executando quatro workers no gateway (-w 4). Cada worker é um processo Python independente, com o seu próprio GIL, capaz de servir requisições em paralelo de verdade. Esta é a escala vertical do processo: mais workers no mesmo container.

A limitação seguinte, identificada com o load test, é o estado: dois workers a tentarem gravar o pedido na mesma estrutura em memória colidem. Por isso o estado foi externalizado para Postgres e o id do pedido passou a ser gerado pela base de dados, removendo a corrida.

3.3 Parte B — Escala em microserviços

Replicámos o serviço de pagamento em três instâncias declaradas explicitamente no docker-compose.yml (payment1, payment2, payment3), cada uma com o seu INSTANCE_ID. Optámos por declarar as réplicas com nomes distintos, em vez de usar docker compose --scale, porque torna o efeito do balanceador observável: cada instância tem um hostname próprio e responde com o seu ID.

O load balancer é manual e vive dentro do monólito (monolith._next_payment_url). É um itertools.cycle sobre a lista de URLs lidas da variável de ambiente PAYMENT_SERVICE_URLS, protegido por um threading.Lock para ser seguro entre threads do mesmo worker. A estratégia é round-robin estrita, registando em log o destino escolhido para cada chamada.

```
PAYMENT_SERVICE_URLS=
    http://payment1:5001/payment,
    http://payment2:5001/payment,
    http://payment3:5001/payment

def _next_payment_url():
    with _rr_lock:
        return next(_rr_cycle)
```

Para evitar que uma falha do pagamento propague para o cliente, a chamada está envolvida em tenacity: três tentativas com backoff exponencial e timeout curto. Combinado com a replicação, se uma instância estiver indisponível, o retry seguinte vai cair noutra réplica.

3.4 Parte C — Testes de carga

O loadtest.py dispara N requisições com M workers concorrentes contra POST /api/orders, medindo latência (min, média, mediana, p95, p99, máx), taxa de sucesso, distribuição de códigos HTTP e contagem de respostas por instância de pagamento. A última métrica é o que torna o balanceamento visível: se o round-robin estiver a funcionar, as três réplicas devem aparecer com contagens aproximadamente iguais.

```
python loadtest.py -n 100 -c 10      # baseline
python loadtest.py -n 500 -c 100     # stress
```

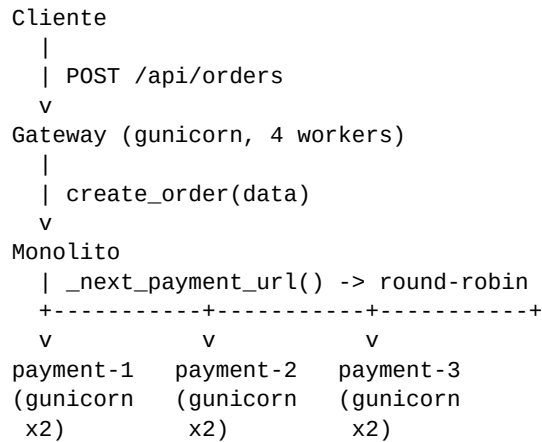
O que procurar nos resultados: erros ou estados não-200 indicam saturação de workers ou timeouts; um plateau de throughput quando se aumenta a concorrência indica que se atingiu o gargalo; p95/p99 muito acima da mediana indica latência de cauda — tipicamente uma dependência lenta a montante ou contention numa lock.

3.5 Parte D — Análise comparativa

Critério	Monólito (1 processo)	Microserviços escalados
Escalabilidade	Limitada pelo processo único; só escala verticalmente.	Cada serviço escala independentemente; gateway e pagamento podem ser replicados sem mudar o código.
Complexidade	Muito baixa: um único deploy, um único log.	Maior: várias instâncias, balanceador, Postgres, idempotência, observabilidade por instância.
Performance	Boa para baixa carga; cai rapidamente quando o pagamento bloqueia o worker.	Throughput agregado maior; latência média semelhante mas com melhor p95/p99 sob carga.
Resiliência	Frágil: uma única falha derruba tudo.	Maior: retry com backoff reencaminha para outra réplica; queda de uma réplica do pagamento não derruba o sistema.

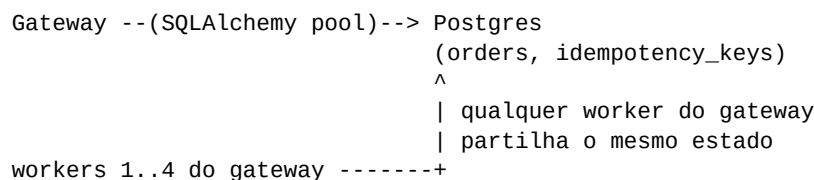
4. Diagramas

4.1 Fluxo com load balancer



As respostas trazem 'instance' -> log no monolito

4.2 Replicação e persistência



5. Demonstração: com e sem escala

Sem escala — uma única réplica do pagamento:

```

PAYMENT_SERVICE_URLS=http://payment1:5001/payment \
docker compose up --build
python loadtest.py -n 500 -c 100

```

Com escala — três réplicas (default):

```

docker compose up --build
python loadtest.py -n 500 -c 100

```

Resultado esperado: o cenário com escala mantém o throughput mais estável quando se aumenta a concorrência e apresenta p95/p99 menores, porque três instâncias absorvem em paralelo a chamada de pagamento em vez de uma só. O bloco "distribuição por instância" impresso pelo load test mostra a contagem de respostas por réplica, confirmando que o round-robin distribui o tráfego.

6. Conclusão

O exercício mostra de forma prática a diferença entre escalar verticalmente um processo e escalar horizontalmente um serviço stateless atrás de um balanceador. As intervenções foram incrementais e cada uma resolveu um gargalo concreto: gunicorn para deixar de servir requisições uma a uma, Postgres para tirar o estado de memória, idempotência para retries não duplicarem pedidos, timeout e retry para a chamada de pagamento não propagar lentidão, e por fim o round-robin para deixar de ter um único ponto a processar pagamentos.

Os limites visíveis: a base de dados continua a ser um nó vertical e seria o próximo gargalo se o tráfego continuasse a crescer; o balanceador vive dentro do monólito, o que é simples mas não é o que se faz em produção (tipicamente seria nginx, HAProxy ou um service mesh à frente das réplicas). Estes seriam os passos naturais de continuação.